



ONTWERPDOCUMENT

V2.12

Hogeschool Leiden
Informatica

Daymon Walter, Jona Wessel, Lukas Hoogendam & Renzo
Steller

Inhoud

Inleiding.....	2
Datamodel en synthetische data	3
Datamodel	3
Ster diagram	4
Architectuur	5
Voordelen van dit ontwerp.....	5
Alarm-ster:.....	5
Enquete-ster:	5
Shared dimension	5
Synthetische data	6
Data referentie architectuur	8
Tooling & Technologie	9
TOGAF-model:	9
Roadmap en implementatieplanning	12
Afweging Oplossingsrichtingen.....	14
Ethics by Design & Security	15
Afstemming en tooling	16
Proof-of-Concept (PoC)	21
Bronnenlijst	24
Versiegeschiedenis	25

Inleiding

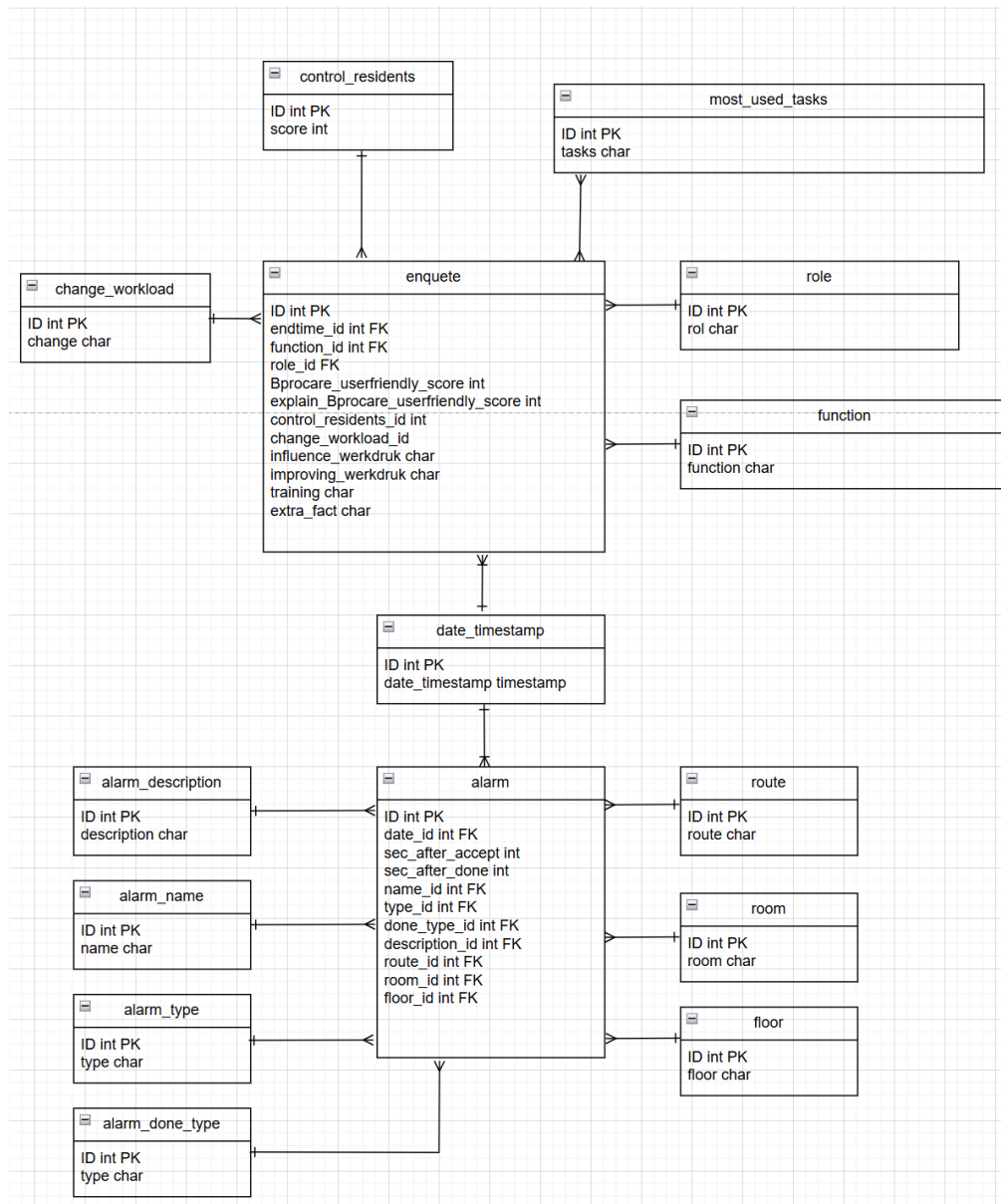
Dit document geeft een overzicht van het ontwerp voor het dashboard en chatbot die wij gaan maken voor zorgbalans.

Het dashboard zal ervoor zorgen dat gebruikers gemakkelijk inzicht krijgen van de relevante data. De chatbot daarentegen helpt met vragen beantwoorden.

Datamodel en synthetische data

Datamodel

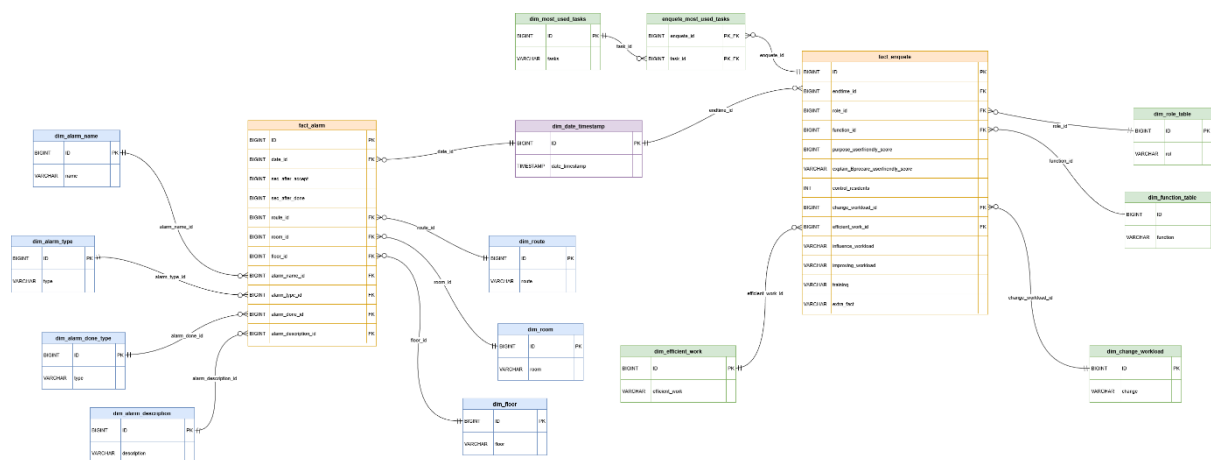
Het eerste wat wij hebben gemaakt is een ERD (entity relationship diagram). Hierin staat hoe alle data met elkaar verbonden is. Zo ziet het ERD eruit:



Je ziet dat het ERD bestaat uit 2 onderdelen. De ene bestaat uit alle meldingen in het nieuwe systeem en de andere bestaat uit de resultaten. Sommige onderdelen staan los, omdat ze meerdere keren hetzelfde benoemen. Op deze manier noem je de data maar 1 keer en is de database genormaliseerd. Een genormaliseerde database heeft minder opslag nodig en je kan de data sneller updaten.

Ster diagram

Vervolgens hebben we een sterdiagram gemaakt. Deze ziet er zo uit:



De tabellen alarm en enquête zijn feitentabellen en de tabellen daaromheen dimensies.
Date_timestamp wordt zowel gebruikt in alarm en enquête (shared dimension).

Architectuur

- **Constellation Schema** (Melkwegstelsel): Dit is een uitbreiding van het klassieke sterdiagram met meerdere feitentabellen die dimensietabellen delen (zoals `date_timestamp`).

Tabel	Doel	Meetwaarden
fact_alarm	Registreert alarm events	sec_after_accept, sec_after_done (Responstijden)
Fact_enquete	Slaat enquêteresultaten op	purpose_userfriendly_score, control_residents (Scores/tellingen)

Voordelen van dit ontwerp

1. Denormalisatie: Eenvoudige query's voor rapportages/dashboards
2. Flexibiliteit: Dimensies kunnen onafhankelijk worden uitgebreid
3. Performance: Geoptimaliseerd voor OLAP (Online Analytical Processing)

Alarm-ster:

- ***dim_route, dim_room, dim_floor*** - Locatie dimensies
- ***dim_alarm_name, dim_alarm_type, dim_alarm_done_type, dim_alarm_description*** - Alarm classificatie
-

Enquete-ster:

- ***dim_role_table, dim_function_table*** - Respondent informatie
- ***dim_change_workload, dim_efficient_work*** - Werkdruk evaluatie
- ***dim_most_used_tasks*** - Meest gebruikte taken (via junction tabel)

Shared dimension

- ***dim_date_timestamp*** - Gedeelde tijdsdimensie die analyses over beide feitentabellen mogelijk maakt (bv. trends over tijd vergelijken)

Synthetische data

Wat is synthetische data?

“Synthetic data is information that's artificially manufactured rather than generated by real-world events. It's created algorithmically and is used as a stand-in for test data sets of production or operational data to validate mathematical models and train machine learning (ML) and deep learning models.”

(Hashemi-Pour et al., 2024b)

Gaan wij dit gebruiken?

Zorgbalans heeft geen data meer van het oude systeem Sensara. We willen graag een vergelijking maken tussen Sensara en Bprocare. Vandaar dat wij een dataset gaan creëren voor dat systeem, maar we kunnen niet zomaar een dataset genereren van Sensara. Dit komt omdat we niet weten hoe de data eruitzag. Vandaar dat we een vragenlijst hebben gemaakt voor de opdrachtgever, zodat de synthetische data iets meer overeenkomt met de missende data.

Dit is de vragenlijst:

1. Wat was de naam van het oude systeem?
2. Hoe zagen de alarmen eruit? Passief en actief? "uit bed melding" en "uit kamer melding"? Wat voor soort meldingen zaten hierin?
3. Werden meldingen geaccepteerd en afgerond op je telefoon net zoals nu? Of moest je iets anders doen?
4. Sinds wanneer was het nieuwe systeem ingegaan?

Hoe gaan we deze data genereren

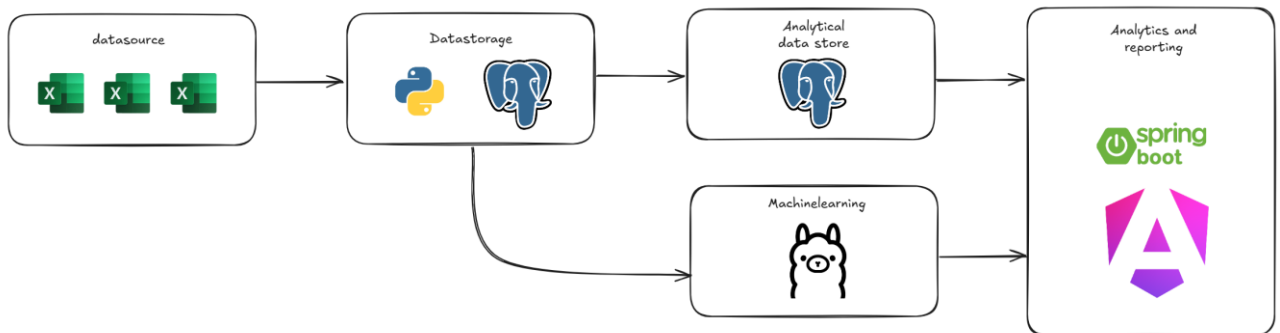
De opdrachtgever heeft de vragenlijst beantwoord (resultaat benoemen). Met deze data gaan we een python script schrijven waarin we faker en random gebruiken. We hebben alle kolomnamen hetzelfde gehouden als in de originele database van BproCare. Zorgbalans is in mei 2025 overgestapt naar BproCare, dus de data is gebaseerd op de eerste 4 maanden van 2025. De tijd tot het accepteren van een melding is vergelijkbaar, dus die we hebben relatief hetzelfde nagebootst. De tijd tot het kunnen helpen van de patiënt is alleen vergelijkbaar met de actieve meldingen, omdat er tijdens het oude systeem nog geen gebruik werd gemaakt van het uitchecken met een NFC Sticker. Alle afmeldingen ging via de telefoon. We hebben de tijd van de meldingen in het oude systeem, Sensara, dus nagebootst op de maximale, minimale en gemiddelde tijd van de actieve meldingen van nu. In de tijd van het oude systeem waren er maar 4 soorten alarmen en is er ook in de data rekening gehouden met hoe vaak deze alarmen

voorkomen. De kamernummers en afdelingen zijn ook toegepast net zoals die terug te vinden zijn in het huidige systeem.

Waar gaan we deze data opslaan

De dataset bevat een variabele die tussen de originele en synthetische data verschilt. Hierdoor zijn de twee datasets te onderscheiden. Vandaar dat we de data opslaan bij de andere alarmen in de database.

Data referentie architectuur



Hierboven zie je de referentie architectuur voor ons project, een webapplicatie met daarin dashboards en een chatbot.

Datasource

De datasource bestaat uit 3 Excel bestanden. De alarmdata, de enquêteresultaten en de synthetische data.

Datastorage

We gaan gebruik maken van een pythonscript om alle data in een postgresdatabase te zet.

Analytical data store

Dit is een sterdiagram. De gegevens zullen in één keer zo doorgestuurd naar het dashboard.

Machinelearning

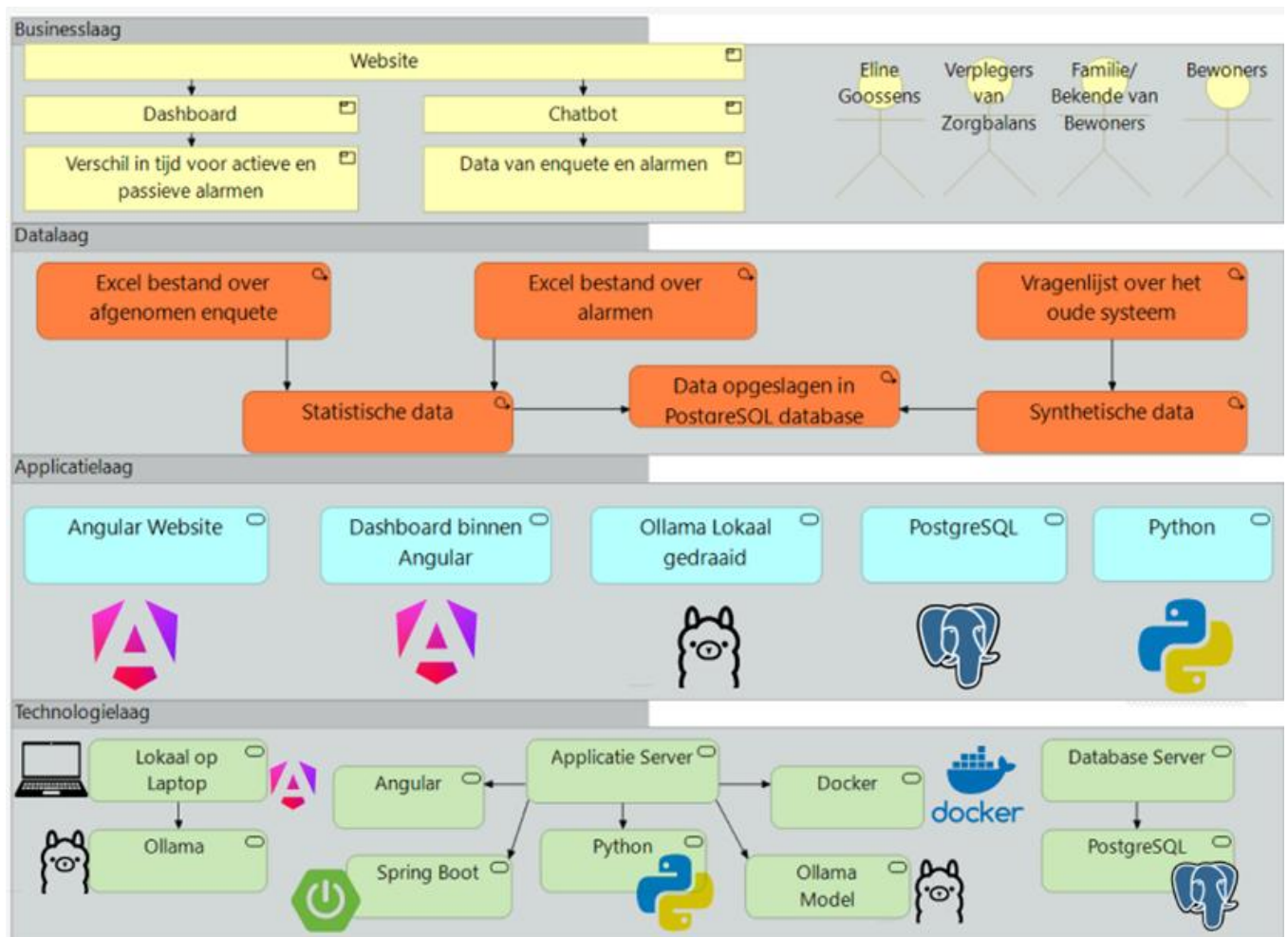
We gaan een ollama chatbot model trainen om antwoorden te geven op vragen van de opdrachtgever.

Analitical and reporting

Wij zijn van plan om een webapplicatie te maken waar zorgbalans makkelijk beide oplossingen kan zien.

Tooling & Technologie

TOGAF-model:



Businesslaag

Doel: Inzicht geven in het gebruik van het Bprocare systeem, om het efficiënter te maken.

We bouwen een website, waarin het dashboard en de chatbot toegankelijk is voor de gebruiker. Het dashboard laat data zien over de passieve en actieve alarmen met behulp van grafieken. De chatbot heeft data uit de enquête en de data over de alarmen en kan tekstueel een uitgebreider antwoord bieden. De website zal gebruikt worden door Eline Goossens en andere verplegers van Zorgbalans, maar zal uiteindelijk ook invloed hebben op de bewoners en hun familie/bekenden.

Data laag

We hebben twee excel bestanden met data, één bestand over de duur van alarmen en waar deze plaatsvinden en één over de afgenomen enquête. Daarnaast gaan we met behulp van een vragenlijst over het oude systeem synthetische data genereren. De statistische en synthetische data worden opgeslagen in de PostgreSQL database.

Applicatielaag

Angular Website

Dit is de front-end van de applicatie. Hierdoor kunnen we het dashboard en de chatbot binnen één applicatie bouwen.

Dashboard binnen Angular:

Het visualiseert de data van de alarmen uit de database. Dit creëert een gebruiksvriendelijk overzicht voor de gebruiker.

Ollama

Ollama is een AI/LLM-model. Dit is een AI-model die lokaal gedraaid kan worden, waardoor de gevoelige gegevens niet gelekt zullen worden. De gebruiker kan gemakkelijk met een chatbot communiceren en kan een uitgebreidere uitleg krijgen ten opzichte van een dashboard.

PostgreSQL

Dit is de database waar we alle onze eerder verkregen data naartoe hebben verplaatst. Het communiceert met de Springboot backend.

Python

Om de database te kunnen vullen vanuit de excel bestanden, hebben we een python script geschreven om de data te kunnen transporteren.

Technologielaag

Lokaal op Laptop

Op onze lokale laptop draait Ollama. We trainen deze lokaal zodat gevoelige informatie niet gelekt wordt. Zodra de AI getraind is wordt de nieuwste versie naar de applicatie server gestuurd.

Applicatie Server

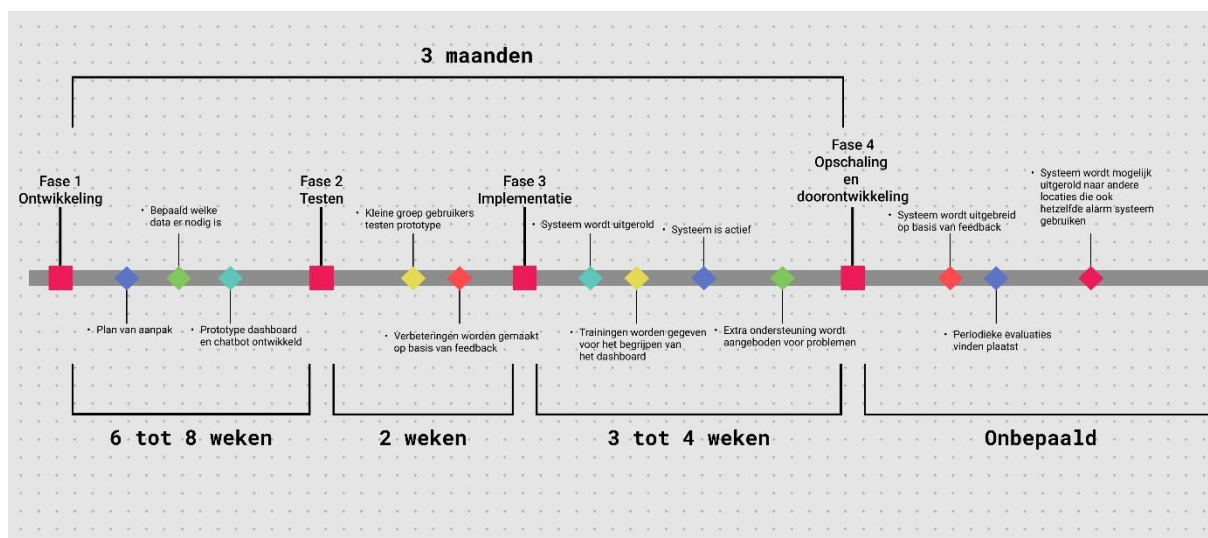
Hier draait de angular website. In de backend gebruiken we springboot die communiceert met de database. In de frontend gebruiken we een angular framework, zodat we een website hebben waarin het dashboard en chatbot in een applicatie zitten. We hebben python gebruikt om de data vanuit excel bestanden naar de database te verplaatsen. Het Ollama model is gekozen om als AI-model te gebruiken, omdat we die lokaal kunnen trainen en binnen een angular project werkt. Daarnaast gebruiken we docker om de verbinding naar de database te laten werken.

Database Server

Alle data bevindt zich in een PostgreSQL database en er kan via een python script data aan worden toegevoegd. De applicatie server vraagt de data hier op via de springboot backend.

Roadmap en implementatieplanning

Deze roadmap legt uit hoe ons systeem ontwikkelt, getest, geïmplementeerd en geschaald kan worden in de praktijk. Zo komt er een duidelijk beeld van de stappen die kunnen gemaakt worden om dit idee tot realisatie te brengen. Door het systeem stap voor stap te ontwikkelen, te testen en uit te rollen, kan worden voorkomen dat het eindproduct niet aansluit bij de praktijk of onvoldoende is voor de gebruikers.



Fase 1. Ontwikkeling (6 tot 8 weken)

In deze fase wordt de technische en inhoudelijke basis van het systeem gemaakt. Eerst worden de benodigde ontwerp stappen gezet (zie het Plan van Aanpak). Daarna wordt er bepaald welke informatie nodig is om inzicht te krijgen in waar het BproCare systeem problemen veroorzaakt of onnodig veel tijd kost. Vervolgens worden de relevante databronnen in kaart gebracht en gekoppeld aan het dashboard. Tegelijk wordt een eerste versie van het dashboard ontworpen waarbij er focus komt op de minimale benodigde functies. Ook wordt een eenvoudige chatbot ontwikkeld die vooraf gedefinieerde vragen kan beantwoorden op basis van de beschikbare data. Aan het einde van deze fase is er een werkend prototype.

Fase 2. Testfase (2 weken)

Tijdens de testfase wordt het prototype getest door een kleine groep eindgebruikers, zoals teamleiders en ondersteunende medewerkers. De nadruk ligt op gebruiksvriendelijkheid, begrijpelijkheid van de visualisaties en betrouwbaarheid van de data. Gebruikers testen of zij met het dashboard en de chatbot daadwerkelijk inzicht krijgen in waar het systeem knelpunten veroorzaakt. Feedback wordt verzameld en verwerkt in verbeteringen. Eventuele fouten in datakoppelingen of onduidelijke visualisaties worden in deze fase opgelost.

Fase 3. Implementatie (3 tot 4 weken)

Na de testfase wordt het systeem uitgerold binnen de afdelingen van de organisatie die relevant zijn. Ze krijgen een korte training of instructie, gericht op het interpreteren van het dashboard en het stellen van effectieve vragen aan de chatbot. Tijdens de implementatie wordt extra ondersteuning aangeboden om vragen te beantwoorden en eventuele problemen snel op te lossen. Dit gebeurt doormiddel van een FAQ (Frequently Asked Questions) en/of IT Medewerker die het systeem helemaal begrijpt. Het systeem wordt nu actief gebruikt voor het signaleren en bespreken van knelpunten van het BproCare systeem in het zorgproces.

Fase 4. Opschaling en doorontwikkeling (doorlopend, start na 3 maanden)

In de opschalingsfase wordt het systeem uitgebreid op basis van gebruikservaringen. Dit kan betekenen dat er extra databronnen worden toegevoegd, visualisaties worden verbeterd of dat de chatbot complexere vragen kan beantwoorden. Ook kan het systeem worden uitgerold naar andere locaties of teams binnen de organisatie. Regelmatige evaluaties zorgen ervoor dat het dashboard en de chatbot blijven aansluiten bij veranderende processen en informatiebehoeften.

Geschatte totale doorlooptijd

De initiële implementatie zal dus rond ongeveer 12 weken (3 maanden) duren. Hierna komen er continu verbeteringen op basis van het gebruik en feedback.

Afweging Oplossingsrichtingen

We hebben onderzoek gedaan naar verschillende oplossingen voor ons dataprobleem. Dit is terug te lezen in ons onderzoekverslag, waar we allereerst hebben gekeken naar alle mogelijkheden om data inzichtelijk te maken. Daarna hebben we dus 4 verschillende dataoplossingen vergeleken op het gebied van gebruikersvriendelijkheid en overzichtelijkheid.

Om de beste oplossing voor dit dataprobleem te vinden, is het van belang om de wensen van de opdrachtgever en de bronnen van data mee te nemen. De opdrachtgever wil inzicht in de data van de enquête en de data over de meldingen van de bewoners.

Dataset 1: Meldingen van de bewoners

Een dashboard is sterk om informatie over het aantal meldingen door de tijd inzichtelijk te maken. Het is ook praktisch bruikbaar voor het management en de teamleiders. Onderzoek toont aan dat dashboards voor data in zorginstanties vaak gebruikt wordt en dat ze kunnen bijdragen aan kwaliteitsverbetering.

Dataset 2: Enquête van de medewerkers

We willen de data uit de enquête kunnen omzetten naar snelle antwoorden op de vragen, zodat de gebruiker data kan opvragen en hier ook een tekstuele uitleg over kan krijgen. De chatbot is bewezen om goed te zijn in het omzetten van tabellen naar statistieken of gesproken uitleg.

Daarnaast moet de dataoplossing toegankelijk zijn voor de gebruiker, omdat zij geen ervaring hebben als data-analist. Een chatbot zorgt ervoor dat de gebruiker met 'gewone woorden' een input kan geven.

We bouwen dus een website waarin we deze 2 oplossingen verwerken om de data op meerdere manieren inzichtelijk te maken. Waarbij het dashboard vooral snel de data inzichtelijk maakt en de chatbot meer verdiepend op de data in kan gaan en ook een inzicht in de data van de medewerkers kan geven. Zo kunnen we kijken welke het beste is voor zorgbalans.

Ethics by Design & Security

Bij het plan van aanpak hebben we aangegeven dat bij data je moet letten op privacy, bias en eerlijkheid. Ook hebben we aangegeven dat we tijdens het ontwikkelen van de software de opdrachtgever erbij gaan betrekken, dat we letten op security.

Anonimiteit en privacy

We hebben de gegevens van de opdrachtgever anoniem binnengekregen. Je kan in de data niet zien om welke bewoner het gaat. Ook kan je niet zien dat de data van zorgbalans is en over welke locatie het gaat. Dit voldoet ook direct aan de privacywetgeving zoals de AVG. Dit zijn de wetten die aangeven hoe je met de gegevens van andere om moet gaan.

Bias/eerlijkheid

We tonen zowel de positieve als negatieve data om een volledig beeld te vormen. Door zowel de positieve en negatieve data te tonen, voorkom je een bias. Dit maakt de oplossing ook transparant en transparantie zorgt ervoor dat je eerlijkere conclusies kan maken.

Opdrachtgever betrekken

We hebben om de twee weken een sprint review met de opdrachtgever. Hierin laten we zien wat we in de afgelopen 2 weken hebben gedaan en we leggen uit waarom we dat hebben gedaan. Zo weet de opdrachtgever wat de stand van zaken is. Zo laten we zien dat we transparant zijn en als de opdrachtgever het ergens niet mee eens is, kan het tijdens het ontwikkelen aangepast worden.

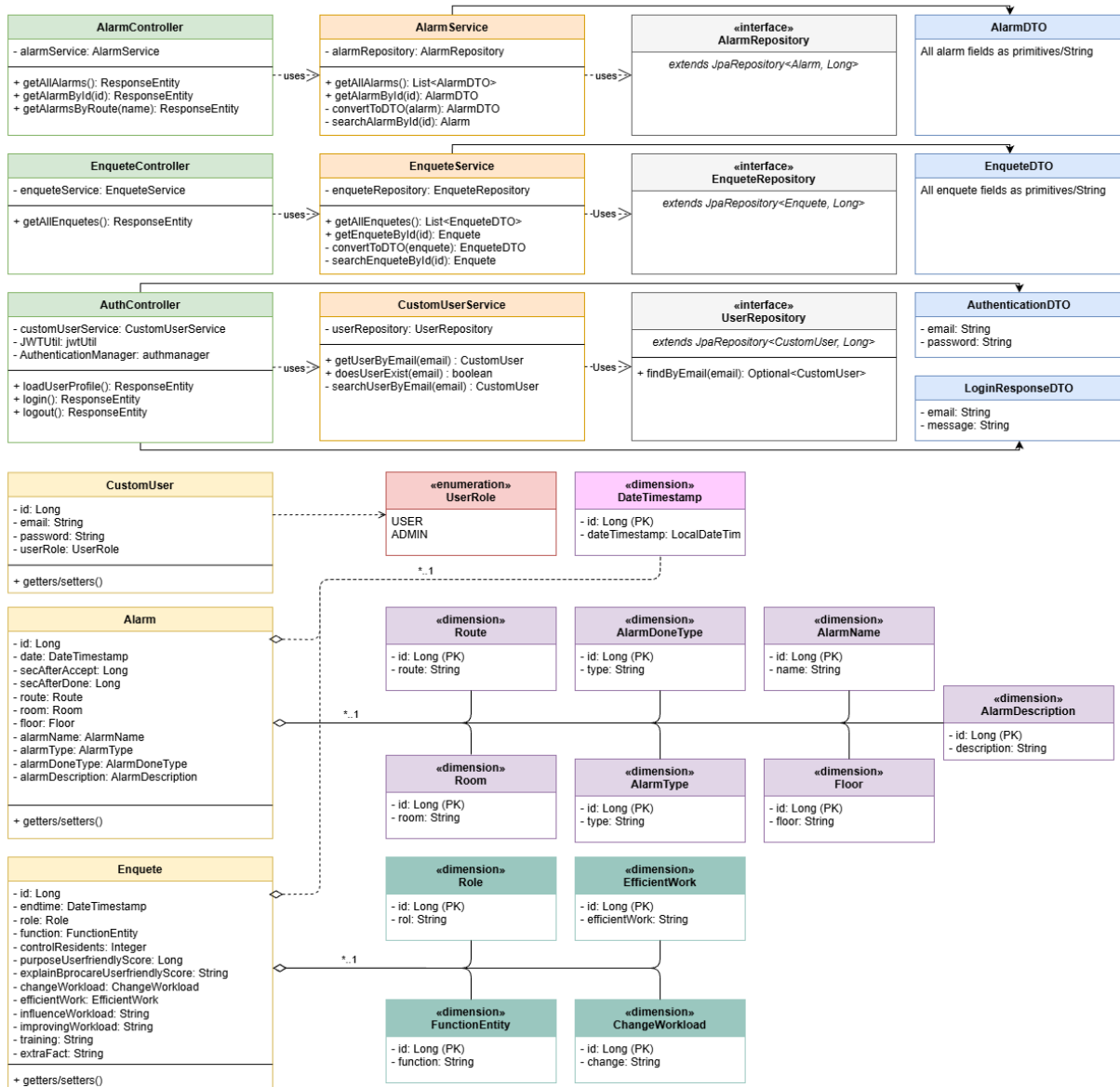
Security

We hebben een eigen omgeving gemaakt zodat de gegevens binnen zorgbalans blijven en niet bij een extern bedrijf terecht komen zoals google of openAI. Veder autoriseren we met de hulp van cookies om te kijken of iemand nog ingelogd is en jwt-tokens om te kijken of de gebruiker nog in een geldige sessie zit en voor toegangsbeheer.

Daarnaast slaan we wachtwoorden op een veilige manier op door ze te hashen. Hashen betekent dat je door een wiskundige functie doet om een onherkenbare en unieke waarde te maken. Deze slaan we op zodat je niet kan zien wat iedereen wachtwoord is.

Afstemming en tooling

Miz17 Backend - Class Diagram



Om data door te kunnen sturen naar het dashboard en de chatbot hebben we een api nodig. Hiervoor gebruiken we springboot api. Hierboven is te zien hoe elke controller, service, repository en model met elkaar verbonden is. We hebben een identieke flow voor Alarm en Enquete. En daarnaast is er een flow voor de authenticatie, omdat we niet willen dat er ongeautoriseerde personen zomaar verzoeken kunnen doen naar de data.

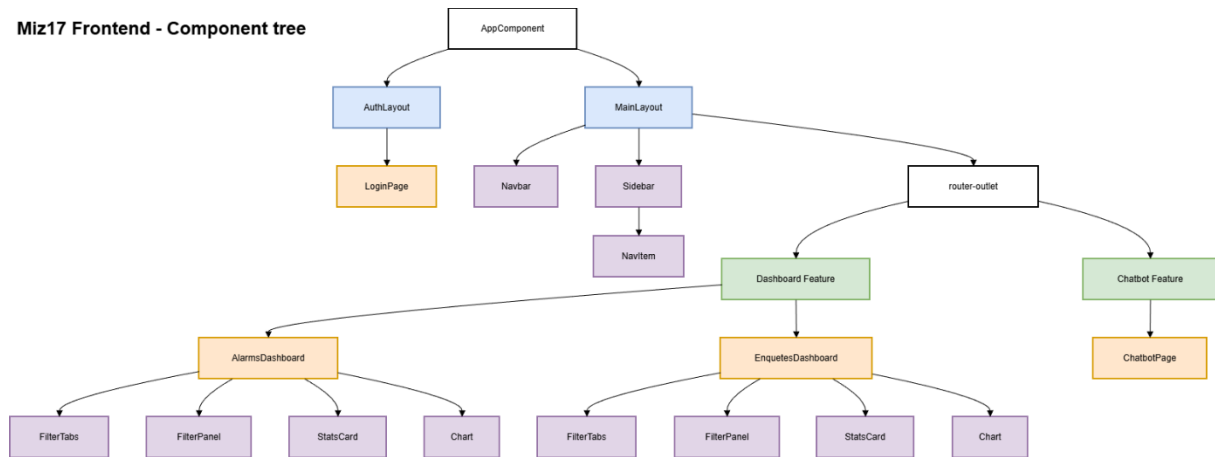
File structuur

```

backend/
├── src/main/java/
│   ├── config/
│   ├──
│   └── controllers/
│       ├── dto/
│       ├── exceptions/
│       ├── filters/
│       ├── models/
│       ├──
│       └── repositories/
│           ├── services/
│           └── utils/
└──
src/main/resources/
    ├── db/
    ├── static/
    └── templates/
  
```

Folder	Beschrijving
config/	Configuratieklassen voor Spring Security, CORS, en database instellingen
controllers/	REST API-controllers die HTTP requests afhandelen
dto/	Data Transfer Objects voor API request/response communicatie
exceptions/	Custom exception klassen en global exception handlers
filters/	Request filters voor validatie en beveiliging
models/	JPA Entity klassen die database tabellen representeren
repositories/	Spring Data JPA repositories voor database operaties
services/	Business logic en service laag klassen
utils/	Hulpklassen voor JWT, cookies en andere utilities
application.properties	Standaard applicatie configuratie
application-dev.properties	Development omgeving configuratie
application-prod.properties	Productie omgeving configuratie
db/	Database migratie scripts (Flyway)
static/	Statische bestanden (CSS, JS, images)
templates/	Server-side templates (Thymeleaf)

Miz17 Frontend - Component tree



We werken met angular dus dat betekent dat we met verschillende componenten werken. Hierboven staat een schets van hoe onze componenten tree eruit ziet. We hebben 2 layouts: AuthLayout en MainLayout. Mainlayout bevat de side/nav bar, en routeroutlet. Routeroutlet kan wisselen tussen de 2 features: dashboard en chatbot.

Blauw = layouts

Groen = features

Orangje = paginas

Paars = herbruikbare components

File structuur

frontend/src/app/

```

├── core/
│   ├── api/
│   ├── guards/
│   ├── interceptors/
│   ├── models/
│   └── services/
├── features/
│   ├── auth/
│   ├── chatbot/
│   └── dashboard/
├── layout/
│   ├── auth-layout/
│   └── main-layout/
└── shared/
    ├── components/
    ├── directives/
    ├── pipes/
    └── services/
  
```

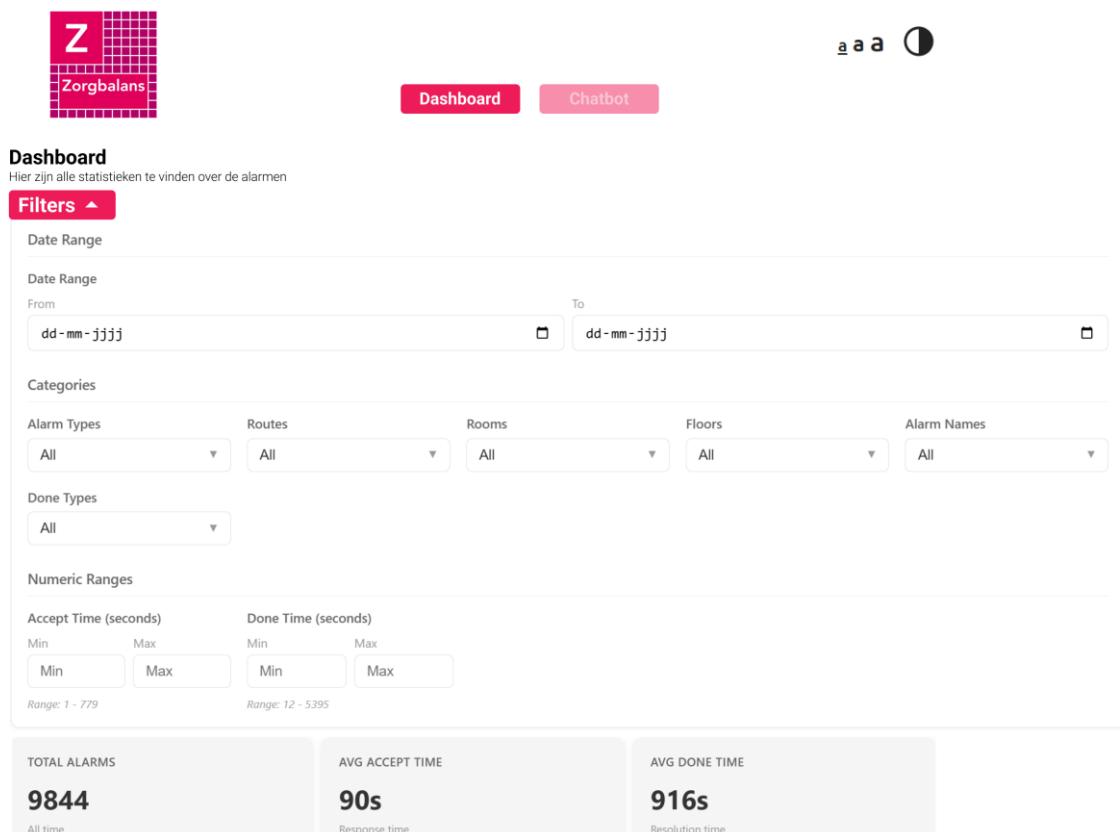
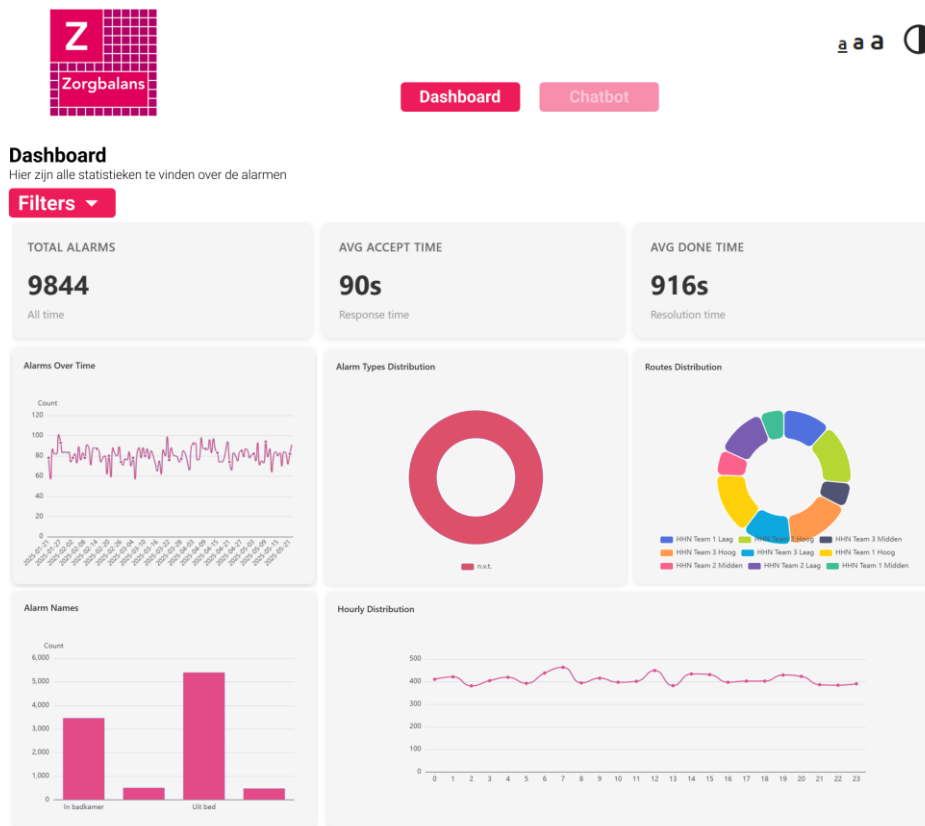
Folder	Beschrijving
core/	Kern functionaliteit die eenmalig wordt geladen
core/api/	API-configuratie en base URL-instellingen
core/guards/	Route guards voor authenticatie en autorisatie
core/interceptors/	Http-interceptors voor token handling en error handling
core/models/	Core datamodellen en interfaces
core/services/	Singleton services (bijv. AuthService)
features/	Feature modules met lazy loading
features/auth/	Login pagina en authenticatie logica
features/chatbot/	Chatbot interface en AI-communicatie
features/dashboard/	Analytics dashboards voor alarmen en enquêtes
layout/	Layout componenten
layout/auth-layout/	Layout voor unauthenticated pagina's
layout/main-layout/	Hoofdlayout met navbar en sidebar
shared/	Herbruikbare componenten en utilities
shared/components/	Gemeenschappelijke UI componenten (Loader, Navbar)
shared/directives/	Angular directives
shared/pipes/	Angular pipes voor data transformatie
shared/services/	Gedeelde services

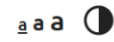
Endpoints

Controller	Method	Endpoint	Beschrijving
AuthController	GET	/auth/profile	Laad gebruikersprofiel
	POST	/auth/login	Inloggen
	POST	/auth/logout	Uitloggen
AlarmController	GET	/analytics/alarm	Alle alarmen ophalen
	GET	/analytics/alarm/{id}	Alarm op ID ophalen
EnqueteController	GET	/analytics/enquete	Alle enquêtes ophalen
	GET	/analytics/enquete/{id}	Enquête op ID ophalen
ChatController	POST	/chatbot	Chatbericht versturen

Voor alle endpoints moet je authorised zijn, behalve auth/login.

Proof-of-Concept (PoC)





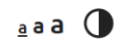
Dashboard

Chatbot

Chatbot

Hier kan je vragen stellen om meer inzicht te krijgen in de alarmen en enquêtes

Type hier je vraag voor de chatbot...



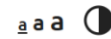
Dashboard

Chatbot

Chatbot

Hier kan je vragen stellen om meer inzicht te krijgen in de alarmen en enquêtes

Welke dag had de meeste alarmen?



Dashboard

Chatbot

Chatbot

Hier kan je vragen stellen om meer inzicht te krijgen in de alarmen en enquêtes

Welke dag had de meeste alarmen?
Just now

De dag met de meeste alarmen was 26 januari 2025. Er waren die dag 101 alarmen.
Just now

Type hier je vraag voor de chatbot... 

<https://www.figma.com/proto/M0ALZi5Woxr8NSqccsDH1J/PoC-Zorgbalans>

Bronnenlijst

Hashemi-Pour, C., Yasar, K., & Laskowski, N. (2024b, december 26). What is synthetic data? Examples, use cases and benefits. Search CIO.

<https://www.techtarget.com/searchcio/definition/synthetic-data>

Versiegeschiedenis

Datum	Auteur	Wijziging
08-12-25	Renzo Steller	Document aangemaakt en inleiding geschreven.
09-12-25	Renzo Steller	ERD gemaakt en toegevoegd aan het document.
10-12-25	Renzo Steller	Erd beschrijving en de synthetische data geschreven.
16-01-26	Renzo Steller	Ik heb een opzet gemaakt voor synthetische data voor Jona, ik heb datareferentie architectuur verbeterd en afgemaakt.
16-01-26	Jona Wessel	Synthetische data afgemaakt met uitleg van proces en resultaten.
17-01-26	Renzo Steller	Ik heb ethics by design afgemaakt.
19-01-26	Jona Wessel	Afweging oplossingsrichtingen gemaakt.
21-01-2026	Renzo Steller	Ik heb alle punten die gedaan zijn gecontroleerd.
22-01-26	Jona Wessel	Slordigheid- en spelfouten verbetert.